C++ Lab Assignment: Loops

Department of Computer Science & Engineering, IUST Awantipora

Prepared by Dr. Sajaad Ahmed Lone

Course details and instructions

This assignment gives you 44 graded practice programs (15 compulsory) covering every C++ loop type: `for`, `while`, `do-while`, nested loops, loop control (`break`, `continue`) and the range-based `for`.

Learning objectives

By the end of this lab you should be able to:

1. Write `for`, `while` and `do-while` loops and choose the right one for a given problem.
2. Trace a loop by hand: initial value, condition check, body, update.
3. Use nested loops to print patterns and process 2-D data.
4. Control loop flow with `break` and `continue`, and avoid infinite loops.
5. Iterate over arrays, strings and vectors with the range-based `for` loop.

Instructions to students

- Attempt every question marked **(Compulsory)**. Do as many others as you can; they count toward lab performance.
- Use only the loop type named in each part, unless the question says otherwise.
- Every program must start with a comment block: your name, enrolment number, question number and date.
- Use meaningful variable names and indent your code consistently.
- Test each program with the sample input, then with at least two inputs of your own, including an edge case (0, 1, a negative number).
- Do not copy code from classmates or the internet. Identical submissions get zero marks.
- Difficulty levels: **[E]** easy, **[M]** medium, **[H]** hard.

Quick reference: loop syntax

C++ has three basic loops plus the range-based `for`. Choose by what you know before the loop starts.

Loop	Condition checked	Runs at least once?	Use when
`for`	Before each iteration	No	The number of iterations is known
`while`	Before each iteration	No	You repeat until a condition changes (unknown count)

Loop	Condition checked	Runs at least once?	Use when
`do-while`	After each iteration	Yes	The body must run once first (menus, input validation)
Range-based `for`	Automatically, per element	No (if empty)	You visit every element of an array, string or vector

```
// 1. for loop
for (int i = 1; i <= 5; i++) {
    cout << i << " ";          // 1 2 3 4 5
}

// 2. while loop
int n = 1234, digits = 0;
while (n != 0) {
    n /= 10;
    digits++;                  // digits = 4
}

// 3. do-while loop
int choice;
do {
    cout << "Enter 0 to exit: ";
    cin >> choice;
} while (choice != 0);

// 4. range-based for loop (C++11)
int marks[] = {78, 85, 92};
for (int m : marks) {
    cout << m << " ";
}

// 5. loop control
for (int i = 1; i <= 10; i++) {
    if (i == 3) continue;      // skip 3
    if (i == 8) break;         // stop at 8
    cout << i << " ";          // 1 2 4 5 6 7
}
```

Compile with `g++ -std=c++17 -Wall program.cpp -o program`. The `-Wall` flag warns about common loop mistakes such as unused or uninitialised variables.

Part A: the for loop

Use a `for` loop in every program in this part.

A1. [E] Multiplication table (Compulsory). Read an integer n and print its table from 1 to 10.

```
Input:  7
Output: 7 x 1 = 7
        7 x 2 = 14
        ...
        7 x 10 = 70
```

A2. [E] Sum and average. Read N, then N integers. Print their sum and average (to 2 decimal places).

```
Input:  5
        10 20 30 40 50
Output: Sum = 150, Average = 30.00
```

A3. [E] Factorial (Compulsory). Read n (0 ≤ n ≤ 20) and print n!. Use `long long` and explain in a comment why `int` is not enough.

```
Input:  10
Output: 10! = 3628800
```

A4. [M] Fibonacci series. Print the first n terms of the Fibonacci series.

```
Input:  8
Output: 0 1 1 2 3 5 8 13
```

A5. [M] Prime check (Compulsory). Read n and report whether it is prime. Loop only up to √n (use `i * i <= n`).

```
Input:  97          Input:  91
Output: 97 is prime  Output: 91 is not prime (divisible by 7)
```

A6. [M] Series sum. Read n and compute S = 1 + 1/2 + 1/3 + … + 1/n, printed to 4 decimal places.

```
Input:  5
Output: S = 2.2833
```

A7. [M] Count down by steps. Print all multiples of 5 from 100 down to 0, using a decrementing loop variable.

```
Output: 100 95 90 ... 5 0
```

Part B: the while loop

Use a `while` loop in every program in this part. These problems repeat an unknown number of times.

B1. [E] Count and sum digits (Compulsory). Read a positive integer and print how many digits it has and their sum.

```
Input:  40721
Output: Digits = 5, Sum = 14
```

B2. [E] Reverse a number. Read an integer and print it reversed.

```
Input:  12345
Output: 54321
```

B3. [M] Palindrome number (Compulsory). Check whether a number reads the same backwards. Reuse your logic from B2.

```
Input:  1221        Input:  1231
Output: Palindrome   Output: Not a palindrome
```

B4. [M] Armstrong number. A 3-digit number is an Armstrong number if the sum of the cubes of its digits equals the number ($153 = 1^3 + 5^3 + 3^3$). Check a given number.

```
Input:  153          Input:  154
Output: Armstrong    Output: Not Armstrong
```

B5. [M] GCD by Euclid's method (Compulsory). Read a and b. Repeat `r = a % b; a = b; b = r;` while b ≠ 0. Print the GCD and the LCM (LCM = original a × original b / GCD, so save the inputs first).

```
Input:  48 18
Output: GCD = 6, LCM = 144
```

B6. [M] Decimal to binary. Convert a non-negative decimal integer to binary using repeated division by 2.

```
Input:  13
Output: 1101
```

B7. [H] Sentinel-controlled input. Keep reading marks until the user enters −1. Then print the count, highest, lowest and average. Handle the case where −1 is the very first input.

```
Input:  67 89 45 92 -1
Output: Count = 4, Highest = 92, Lowest = 45, Average = 73.25
```

Part C: the do-while loop

Use a `do-while` loop in every program in this part. Each problem needs its body to run at least once.

C1. [E] Input validation (Compulsory). Ask for a mark between 0 and 100. Keep asking until a valid value is entered, then print it.

```
Enter marks (0-100): 120
Invalid! Enter marks (0-100): -5
Invalid! Enter marks (0-100): 87
Accepted: 87
```

C2. [M] Menu-driven calculator (Compulsory). Show a menu repeatedly until the user chooses Exit. Use a `switch` inside the loop and guard against division by zero.

```
1. Add  2. Subtract  3. Multiply  4. Divide  5. Exit
Enter choice: 1
Enter two numbers: 8 5
Result = 13
1. Add  2. Subtract  3. Multiply  4. Divide  5. Exit
Enter choice: 5
Goodbye!
```

C3. [M] Number-guessing game. Fix a secret number between 1 and 100 (or generate one with `rand() % 100 + 1`). Let the user guess, reply "Too high" or "Too low", and print the number of attempts when correct.

```
Guess: 50   Too high
Guess: 25   Too low
Guess: 37   Correct! You took 3 attempts.
```

C4. [M] Sum until zero. Keep reading integers and adding them until 0 is entered. Print the sum of the positive and negative numbers separately.

```
Input:  5 -3 8 -2 0
Output: Positive sum = 13, Negative sum = -5
```

C5. [M] Compare with while. Write the digit-sum program from B1 using `do-while`. Run both versions with input 0. In a comment, explain why the outputs differ and which loop is correct for 0.

Part D: nested loops and patterns

Use an outer loop for rows and an inner loop for columns. Read the number of rows n from the user; the samples use n = 5 or n = 4.

D1. [E] Right-angled triangle of stars (Compulsory).

```
*
* *
* * *
* * * *
* * * * *
```

D2. [E] Number triangle.

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

D3. [M] Pyramid (Compulsory). Print leading spaces with one inner loop and stars with another.

```
    *
   ***
  *****
 *******
*********
```

D4. [M] Floyd's triangle. Numbers continue across rows.

```
1
2 3
4 5 6
7 8 9 10
```

D5. [M] Hollow square. Print stars only on the border of an n × n square.

```
* * * * *
*       *
*       *
*       *
* * * * *
```

D6. [H] Diamond. Print the pyramid from D3 followed by its inverted copy (without repeating the middle row).

```
  *
 ***
*****
 ***
  *
```

(shown for n = 3)

D7. [M] Multiplication grid (Compulsory). Print the tables from 1 to 10 as a 10 × 10 grid, with columns aligned using `setw(4)` from `<iomanip>`.

D8. [H] Primes in a range. Read a and b and print every prime between them. The outer loop walks through the range; the inner loop tests divisibility.

```
Input:  10 30
Output: 11 13 17 19 23 29
```

D9. [H] Matrix multiplication. Read two 3 × 3 matrices and print their product using three nested loops.

Part E: break, continue and loop control

Any loop type may be used. `break` leaves the innermost loop immediately; `continue` skips to its next iteration.

E1. [E] Skip multiples (Compulsory). Print the numbers 1 to 50, skipping every multiple of 3 or 7 with `continue`.

```
Output: 1 2 4 5 8 10 11 13 16 17 19 20 ...
```

E2. [E] First negative. Read up to 10 integers. Stop reading with `break` as soon as a negative number appears, and print its position.

```
Input:  4 9 12 -3 7
Output: First negative number -3 found at position 4
```

E3. [M] Linear search (Compulsory). Read an array of N integers and a key. Search with a loop and `break` when found; otherwise print "Not found".

```
Input:  6
        23 45 12 67 34 89
        67
Output: 67 found at index 3
```

E4. [M] Controlled infinite loop. Write `while (true)` that reads numbers and prints their squares. Exit with `break` when the user enters 0. Explain in a comment why the loop is not truly infinite.

E5. [H] Breaking out of nested loops. Find the first pair (i, j) with $1 \le i, j \le 20$ such that $i^2 + j^2 = 225$. Show two ways to stop both loops: a `bool found` flag checked by the outer loop, and putting the loops in a function that uses `return`.

```
Output: First pair: i = 9, j = 12
```

E6. [M] Find the bug. Each loop below is wrong. Type it in, describe what happens, then fix it.

```
// (a) Should print 1 to 10
int i = 1;
while (i <= 10)
    cout << i << " ";
    i++;

// (b) Should print 10 down to 1
for (int i = 10; i >= 1; i++)
    cout << i << " ";

// (c) Should sum 1 to 100
int sum;
for (int i = 1; i <= 100; i++);
    sum += i;
```

Part F: the range-based for loop

Use `for (auto x : container)` in every program in this part (C++11 or later). Use a reference, `auto&`, when you need to change the elements.

F1. [E] Array statistics (Compulsory). Store 8 marks in an array. Print the total, maximum and minimum using a range-based loop.

```
Marks:  72 85 64 90 58 77 81 69
Output: Total = 596, Max = 90, Min = 58
```

F2. [E] Count vowels (Compulsory). Read a line with `getline` and count its vowels (upper and lower case) by looping over the `string`'s characters.

```
Input:  Islamic University of Science and Technology
Output: Vowels = 15
```

F3. [M] Modify with references. Store prices in a `vector<double>` and increase every price by 10% in place with `for (auto& p : prices)`. Print before and after. Then remove the `&`, run again and explain in a comment why nothing changes.

F4. [M] Even and odd split. From a `vector<int>`, copy even numbers into one vector and odd numbers into another, then print both.

```
Input:  12 7 9 20 33 46
Output: Even: 12 20 46
        Odd:  7 9 33
```

F5. [M] Rewrite with all loop types. Print the elements of the array {5, 10, 15, 20, 25} four times: with `for`, `while`, `do-while` and range-based `for`. Note which version was shortest and which was easiest to get wrong.

Part G: challenge problems (take-home)

Choose the most suitable loop for each problem and justify your choice in a comment at the top of the program.

G1. [H] Perfect numbers. Print all perfect numbers below 10,000. A perfect number equals the sum of its proper divisors (28 = 1 + 2 + 4 + 7 + 14).

```
Output: 6 28 496 8128
```

G2. [H] Pascal's triangle. Print n rows, computing each entry from the previous one with C(r, k+1) = C(r, k) × (r − k) / (k + 1).

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

G3. [H] Bank ATM simulation. Start with a balance of ₹10,000. Use a `do-while` menu for Balance, Deposit, Withdraw and Exit. Reject withdrawals above the balance, and lock the session after 3 wrong PIN attempts entered with a `for` loop.

G4. [H] Collatz sequence. Read n. While n ≠ 1, set n = n / 2 if n is even, else n = 3n + 1. Print the sequence and its number of steps.

```
Input:  6
Output: 6 3 10 5 16 8 4 2 1  (8 steps)
```

G5. [H] Square root by Newton's method. Read x > 0. Start with g = x and repeat g = (g + x / g) / 2 until two successive guesses differ by less than 0.00001. Print the result and the number of iterations, and compare it with `sqrt(x)`.

```
Input:  2
Output: Newton: 1.41421   Library sqrt: 1.41421
```